

第三次作业：随机、近似与在线算法作业解答

姓名：郑文翔

学号：25S003050

1. Las Vegas、Monte Carlo、Sherwood 算法的差异

Las Vegas 算法：结果一定正确，但是运行时间是随机变量。随机性只影响算法运行时间，不影响答案正确性。例如随机快速排序最终一定得到正确排序，但运行时间依赖随机主元的选择。

Monte Carlo 算法：运行时间通常有确定上界，但是输出结果可能有小概率错误。若单次错误概率为 ε ，则通常可以通过重复运行并取多数结果来降低错误概率。

Sherwood 算法：利用随机化消除算法对特殊输入的敏感性，使算法对不同输入具有较均匀的平均性能。它通常不牺牲正确性，而是避免某些输入总是触发最坏情况。

三者差异可概括如下：

算法类型	正确性	运行时间
Las Vegas	一定正确	随机
Monte Carlo	可能错误	通常有确定上界
Sherwood	通常一定正确	随机化平滑输入差异

2. 随机化歌单重排算法及期望步数

题目要求不断随机选择歌曲，直到所有歌曲都至少被重新放置过一次，同时希望每首歌曲在任意位置出现的概率相同。下面给出一种严谨的实现：利用每首歌**第一次被随机选中的**顺序产生随机排列，并在第一次选中时把它放入一个随机空位置。

设原歌单为 $A[1], A[2], \dots, A[N]$ ，输出歌单为 $B[1], B[2], \dots, B[N]$ 。

算法 1 基于首次出现顺序的随机歌单重排算法

```

1:  $U \leftarrow \{1, 2, \dots, N\}$  ▷ 尚未被重新放置过的歌曲编号集合
2:  $P \leftarrow \{1, 2, \dots, N\}$  ▷ 尚未被占用的输出位置集合
3: while  $U \neq \emptyset$  do
4:   从  $\{1, 2, \dots, N\}$  中等概率随机选择一首歌曲  $s$ 
5:   if  $s \in U$  then
6:     从  $P$  中等概率随机选择一个位置  $j$ 
7:     将歌曲  $A[s]$  放到输出歌单的位置  $B[j]$ 
8:      $U \leftarrow U \setminus \{s\}$ 
9:      $P \leftarrow P \setminus \{j\}$ 
10:  else
11:    歌曲  $s$  已经被重新放置过, 本次选择不改变最终输出
12:  end if
13: end while
14: return  $B$ 

```

均匀性证明: 每次在 U 中第一次出现的新歌曲, 在所有尚未出现的歌曲中是等概率的。因此歌曲第一次出现的顺序是 N 首歌曲的一个均匀随机排列。同时, 每首第一次出现的歌曲被放入尚未占用的位置集合 P 中的一个均匀随机位置。因此每个最终排列出现的概率都是

$$\frac{1}{N!},$$

从而每首歌曲出现在任意一个位置的概率都是

$$\frac{1}{N}.$$

期望步数: 该过程停止时, 意味着 N 首歌曲都至少被随机选中过一次, 因此步数等价于优惠券收集问题。设总步数为 T 。当已经有 k 首不同歌曲被选中过时, 下一步选中新歌曲的概率为

$$p_k = \frac{N - k}{N}.$$

从 k 首增加到 $k + 1$ 首所需期望步数为

$$\frac{1}{p_k} = \frac{N}{N - k}.$$

所以

$$\begin{aligned}
 \mathbb{E}[T] &= \sum_{k=0}^{N-1} \frac{N}{N - k} \\
 &= N \sum_{r=1}^N \frac{1}{r} \\
 &= NH_N.
 \end{aligned}$$

其中 $H_N = 1 + \frac{1}{2} + \cdots + \frac{1}{N}$ 。由于 $H_N = \ln N + \gamma + o(1)$ ，所以

$$\mathbb{E}[T] = NH_N = \Theta(N \log N).$$

3. 均匀课程选择算法

题目中说每个学生选择 M 门课程，并且可以重复。若将一次选课结果理解为**有序序列**，则算法很简单；若将其理解为**无序且允许重复的组合**，则需要用星与隔板方法保证每种组合等概率。下面给出更符合“组合”二字的无序可重复版本。

一个无序可重复课程组合可表示为

$$x_1 + x_2 + \cdots + x_C = M, \quad x_i \geq 0,$$

其中 x_i 表示第 i 门课程被选择的次数。这样的组合数为

$$\binom{C+M-1}{M}.$$

星与隔板表示法中，一共有 M 个星号和 $C-1$ 个隔板，所以共有 $C+M-1$ 个位置。等概率选择其中 $C-1$ 个位置放隔板，即可唯一确定一个向量 $(x_1, x_2, \dots, x_C)$ 。

算法 2 无序可重复课程组合的均匀随机算法

- 1: **for** 每个学生 $s = 1, 2, \dots, N$ **do**
 - 2: 在 $\{1, 2, \dots, C+M-1\}$ 中等概率随机选择 $C-1$ 个位置作为隔板位置
 - 3: 其余 M 个位置作为星号位置
 - 4: 令 x_i 为第 $i-1$ 个隔板与第 i 个隔板之间的星号数
 - 5: 输出课程组合：第 i 门课程选择 x_i 次， $i = 1, 2, \dots, C$
 - 6: **end for**
-

正确性证明：星与隔板给出了如下集合之间的一一对应：

$$\{(x_1, \dots, x_C) : x_1 + \cdots + x_C = M, x_i \geq 0\}$$

与

$$\{\text{从 } C+M-1 \text{ 个位置中选择 } C-1 \text{ 个隔板位置}\}.$$

后者共有

$$\binom{C+M-1}{C-1} = \binom{C+M-1}{M}$$

种可能。算法对每一种隔板位置集合等概率选择，因此每一个无序可重复课程组合被选中的概率均为

$$\frac{1}{\binom{C+M-1}{M}}.$$

所以所有课程组合的概率相同。

如果题目把学生的选择结果理解为有序序列，则也可以直接独立选择：

$$c_i \sim \text{Uniform}\{1, 2, \dots, C\}, \quad i = 1, 2, \dots, M.$$

此时任意有序序列 $(a_1, \dots, a_M)$ 的概率为

$$\left(\frac{1}{C}\right)^M.$$

4. 雇用问题的平均费用

设雇用费用为 x ，面试费用为 y ，共有 n 个候选人。每个人都要面试，所以面试费用为

$$ny.$$

假设候选人以随机顺序出现，且能力互不相同。第 i 个候选人被雇用，当且仅当他是前 i 个候选人中最优秀的。由于前 i 个候选人的相对排名均匀随机，

$$\Pr[\text{第 } i \text{ 个候选人被雇用}] = \frac{1}{i}.$$

令 X_i 表示第 i 个候选人是否被雇用，则

$$\mathbb{E}[X_i] = \frac{1}{i}.$$

总雇用次数为 $X = \sum_{i=1}^n X_i$ 。由期望线性性，

$$\begin{aligned} \mathbb{E}[X] &= \sum_{i=1}^n \mathbb{E}[X_i] \\ &= \sum_{i=1}^n \frac{1}{i} \\ &= H_n. \end{aligned}$$

因此总费用期望为

$$\boxed{\mathbb{E}[\text{总费用}] = ny + xH_n.}$$

5. 用有偏随机过程产生无偏随机比特

这是 von Neumann 方法。

算法 3 由 BIASED-RANDOM 构造无偏随机比特

```

1: loop
2:   调用两次 BIASED-RANDOM, 得到  $(a, b)$ 
3:   if  $(a, b) = (0, 1)$  then
4:     return 0
5:   else if  $(a, b) = (1, 0)$  then
6:     return 1
7:   else
8:     丢弃本轮结果, 重新开始
9:   end if
10: end loop

```

因为

$$\Pr[(0, 1)] = (1 - p)p,$$

并且

$$\Pr[(1, 0)] = p(1 - p),$$

二者概率相等。因此在算法停止的条件下, 输出 0 和输出 1 的概率均为 $1/2$ 。

每一轮停止的概率为

$$q = \Pr[(0, 1) \text{ 或 } (1, 0)] = 2p(1 - p).$$

轮数期望为 $1/q$ 。每轮调用两次 BIASED-RANDOM, 因此期望调用次数为

$$\boxed{\frac{1}{p(1 - p)}}.$$

若一次调用视为一个时间单位, 则期望运行时间为

$$\Theta\left(\frac{1}{p(1 - p)}\right).$$

6. 随机归还帽子的期望正确人数

设共有 n 位顾客。令

$$X_i = \begin{cases} 1, & \text{第 } i \text{ 位顾客拿到自己的帽子,} \\ 0, & \text{否则.} \end{cases}$$

由于帽子随机排列后归还, 第 i 位顾客拿到自己帽子的概率为

$$\Pr[X_i = 1] = \frac{1}{n}.$$

所以

$$\mathbb{E}[X_i] = \frac{1}{n}.$$

设拿到自己帽子的总人数为 $X = \sum_{i=1}^n X_i$, 则

$$\begin{aligned}\mathbb{E}[X] &= \sum_{i=1}^n \mathbb{E}[X_i] \\ &= n \cdot \frac{1}{n} \\ &= 1.\end{aligned}$$

因此期望人数为

$$\boxed{1}.$$

7. 生日悖论

设房间中有 k 个人。所有人生日互不相同的概率为

$$P_{\text{diff}}(k) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365 - k + 1}{365}.$$

因此至少两人生日相同的概率为

$$P_{\text{same}}(k) = 1 - P_{\text{diff}}(k).$$

要求 $P_{\text{same}}(k) \geq 1/2$, 等价于 $P_{\text{diff}}(k) \leq 1/2$ 。

用近似 $1 - x \approx e^{-x}$, 有

$$\begin{aligned}P_{\text{diff}}(k) &= \prod_{i=0}^{k-1} \left(1 - \frac{i}{365}\right) \\ &\approx \exp\left(-\sum_{i=0}^{k-1} \frac{i}{365}\right) \\ &= \exp\left(-\frac{k(k-1)}{2 \cdot 365}\right).\end{aligned}$$

令其等于 $1/2$, 得到

$$\exp\left(-\frac{k(k-1)}{730}\right) = \frac{1}{2}.$$

所以

$$k(k-1) = 730 \ln 2 \approx 505.997.$$

解得 $k \approx 23$ 。精确计算也有

$$P_{\text{same}}(22) < \frac{1}{2}, \quad P_{\text{same}}(23) > \frac{1}{2}.$$

因此至少需要

$$\boxed{23}$$

个人。

8. 随机最小割算法

随机最小割算法解决无向图的最小割问题。给定无向连通图 $G = (V, E)$ ，目标是找到非空真子集 $S \subset V$ ，使

$$\delta(S) = \{(u, v) \in E : u \in S, v \in V \setminus S\}$$

的边数最少，即求 $\min_{\emptyset \neq S \subset V} |\delta(S)|$ 。

算法 4 Karger 随机收缩算法

- 1: 输入无向图 $G = (V, E)$
 - 2: **while** 图中顶点数大于 2 **do**
 - 3: 从当前边集中等概率随机选择一条边 (u, v)
 - 4: 收缩边 (u, v) ，即把 u 和 v 合并成一个超级顶点
 - 5: 删除自环，保留重边
 - 6: **end while**
 - 7: **return** 剩下两个超级顶点之间的边集
-

设最小割大小为 λ 。若某个固定最小割中的边在收缩过程中从未被收缩，则最后输出的就是该最小割。

当图中还有 r 个顶点时，最小度至少为 λ ，所以边数满足

$$|E_r| \geq \frac{r\lambda}{2}.$$

随机选中该最小割中边的概率至多为

$$\frac{\lambda}{|E_r|} \leq \frac{\lambda}{r\lambda/2} = \frac{2}{r}.$$

因此不选中最小割边的概率至少为

$$1 - \frac{2}{r} = \frac{r-2}{r}.$$

从 n 个顶点收缩到 2 个顶点，成功概率至少为

$$\begin{aligned} \prod_{r=n}^3 \frac{r-2}{r} &= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{1}{3} \\ &= \frac{2}{n(n-1)}. \end{aligned}$$

重复运行 $O(n^2 \log n)$ 次并取最小结果，可以以高概率得到真正最小割。

9. k 服务员问题的在线算法及竞争比

题目描述的是图上的 k -server 问题。图为 $G = (V, E)$ ，边长为正，并由最短路定义距离 $d(u, v)$ 。有 k 名服务员，初始位于 k 个顶点。请求序列 $r_1, r_2, \dots, r_m$ 在线到达。每次请求 r_t 到达时，算法必须立即选择一名服务员移动到 r_t ，代价为移动距离。

可采用工作函数算法 (Work Function Algorithm, WFA)。设一个配置 S 表示 k 名服务员所在顶点的集合。两个配置 S, S' 之间的距离 $D(S, S')$ 定义为将 S 中服务员移动到 S' 的最小匹配距离。

定义工作函数

$$W_t(S)$$

为处理完前 t 个请求并最终处于配置 S 的最小总代价。初始时

$$W_0(S) = \begin{cases} 0, & S = S_0, \\ +\infty, & S \neq S_0. \end{cases}$$

对第 t 个请求 r_t ，若配置 S 不包含 r_t ，则 $W_t(S) = +\infty$ ；若 S 包含 r_t ，则

$$W_t(S) = \min_{S'} \{W_{t-1}(S') + D(S', S)\}.$$

算法 5 工作函数算法 WFA

```

1: 当前实际配置  $A \leftarrow S_0$ 
2: 初始化工作函数  $W_0(S_0) = 0$ ，其余配置为  $+\infty$ 
3: for  $t = 1, 2, \dots, m$  do
4:   接收在线请求  $r_t$ 
5:   for 每个服务员配置  $S$  do
6:     if  $r_t \in S$  then
7:        $W_t(S) \leftarrow \min_{S'} \{W_{t-1}(S') + D(S', S)\}$ 
8:     else
9:        $W_t(S) \leftarrow +\infty$ 
10:    end if
11:  end for
12:  选择

```

$$S_t = \arg \min_{S: r_t \in S} \{W_t(S) + D(A, S)\}$$

```

13:   将服务员从配置  $A$  移动到  $S_t$ ，并服务请求  $r_t$ 
14:    $A \leftarrow S_t$ 
15: end for

```

工作函数算法在任意度量空间上是

$$2k - 1$$

竞争的，即对任意请求序列 σ ，有

$$\text{cost}_{\text{WFA}}(\sigma) \leq (2k - 1) \text{OPT}(\sigma) + C,$$

其中 C 为只与初始状态有关的常数。因此竞争比为

$$\boxed{2k - 1}.$$

10. 非二分图加权匹配贪心算法的近似比

设贪心算法得到的匹配为 M ，其权值为

$$z = \sum_{e \in M} w_e.$$

设最优匹配为 M^* ，其权值为

$$z^* = \sum_{e \in M^*} w_e.$$

算法 6 最大权匹配的贪心近似算法

```

1:  $M \leftarrow \emptyset$ 
2: 将所有边按权值从大到小排序
3: for 排序后的每条边  $e = (u, v)$  do
4:   if  $u$  和  $v$  均未被  $M$  中的边匹配 then
5:      $M \leftarrow M \cup \{e\}$ 
6:   end if
7: end for
8: return  $M$ 

```

(1) 线性规划最优值是 z^* 的上界

考虑线性规划：

$$\begin{aligned}
& \min \quad \sum_{v \in V} x_v \\
& \text{s.t.} \quad x_u + x_v \geq w_{uv}, \quad \forall (u, v) \in E, \\
& \quad \quad x_v \geq 0, \quad \forall v \in V.
\end{aligned}$$

设最优值为 Z_{LP} 。任取可行解 x ，对任意 $(u, v) \in M^*$ ，有

$$x_u + x_v \geq w_{uv}.$$

由于 M^* 是匹配，其边互不共享端点，所以

$$\begin{aligned} z^* &= \sum_{(u,v) \in M^*} w_{uv} \\ &\leq \sum_{(u,v) \in M^*} (x_u + x_v) \\ &\leq \sum_{v \in V} x_v. \end{aligned}$$

对最优可行解也成立，因此

$$z^* \leq Z_{\text{LP}}.$$

(2) 构造可行解并证明近似比

设贪心选出的匹配为 $M = \{e_1, e_2, \dots, e_q\}$ 。对每条 $e_i = (u_i, v_i)$ ，记 $w_i = w(e_i)$ 。构造

$$x_{u_i} = w_i, \quad x_{v_i} = w_i,$$

对不属于任何贪心匹配边端点的顶点 v ，令 $x_v = 0$ 。

任取边 $e = (a, b) \in E$ 。若 e 被贪心选中，则其两个端点的变量和为 $2w(e) \geq w(e)$ 。若 e 未被选中，则当贪心算法扫描到 e 时，它一定至少有一个端点已经被某条更早选中的边 e_i 匹配。因为边按权值从大到小扫描，所以

$$w_i \geq w(e).$$

设公共端点为 a ，则

$$x_a = w_i \geq w(e), \quad x_b \geq 0,$$

于是

$$x_a + x_b \geq w(e).$$

因此 x 是线性规划的可行解。

该解的目标值为

$$\begin{aligned} \sum_{v \in V} x_v &= \sum_{e_i = (u_i, v_i) \in M} (x_{u_i} + x_{v_i}) \\ &= \sum_{e_i \in M} (w_i + w_i) \\ &= 2z. \end{aligned}$$

所以

$$Z_{\text{LP}} \leq 2z.$$

又因为 $z^* \leq Z_{\text{LP}}$ ，故

$$z^* \leq 2z.$$

即

$$z \geq \frac{1}{2}z^*.$$

因此贪心算法是最大权匹配问题的

2-近似算法.

11. 加权集合覆盖问题的近似算法

全集为 $U = \{e_1, e_2, \dots, e_n\}$, 集合族为 $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, 每个集合 S_i 有权重 $w_i > 0$ 。目标是选择若干集合覆盖 U , 使总权重最小。

算法 7 加权集合覆盖贪心算法

1: $C \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset$

2: **while** $C \neq U$ **do**

3: 选择集合

$$S_i = \arg \min_{S_j: S_j \setminus C \neq \emptyset} \frac{w_j}{|S_j \setminus C|}$$

4: $\mathcal{A} \leftarrow \mathcal{A} \cup \{S_i\}$

5: $C \leftarrow C \cup S_i$

6: **end while**

7: **return** $\mathcal{A}$

设最优解代价为 OPT 。当还有 r 个元素未覆盖时, 最优解可以覆盖这些元素, 总代价为 OPT , 所以其中至少存在一个集合, 其平均覆盖新元素的代价不超过

$$\frac{\text{OPT}}{r}.$$

贪心算法选择平均代价最小的集合, 所以给每个新覆盖元素分摊的代价也不超过 OPT/r 。

按照元素被首次覆盖的顺序计费, 总费用满足

$$\begin{aligned} \text{ALG} &\leq \text{OPT} \left(1 + \frac{1}{2} + \dots + \frac{1}{n} \right) \\ &= H_n \text{OPT}. \end{aligned}$$

因此近似比为

$$H_n \leq 1 + \ln n.$$

12. 满足三角不等式的旅行商问题近似算法

给定带权完全图 $G = (V, E)$, 边权为 $d(u, v)$, 且满足三角不等式:

$$d(u, v) \leq d(u, w) + d(w, v), \quad \forall u, v, w \in V.$$

算法 8 Metric TSP 的最小生成树近似算法

```

1: 求图  $G$  的一棵最小生成树  $T$ 
2: 从任意顶点出发, 对  $T$  做深度优先遍历, 得到访问序列  $Q$ 
3:  $P \leftarrow$  空序列
4: for 按照顺序扫描  $Q$  中的每个顶点  $v$  do
5:   if  $v$  尚未出现在  $P$  中 then
6:     将  $v$  加入  $P$ 
7:   end if
8: end for
9: 将  $P$  的最后一个顶点连接回第一个顶点, 形成 Hamilton 回路
10: return 该回路
  
```

设最优旅行商回路代价为 OPT 。从最优回路中删去任意一条边可得到一棵生成树, 因此

$$w(T) \leq \text{OPT}.$$

沿最小生成树每条边走两遍可得到遍历所有顶点并回到起点的闭合游走, 代价为

$$2w(T) \leq 2\text{OPT}.$$

由于满足三角不等式, 跳过重复访问的顶点不会增加总代价, 所以最终 Hamilton 回路满足

$$\text{ALG} \leq 2w(T) \leq 2\text{OPT}.$$

因此该算法是

2-近似算法

.

若首次访问顺序为 $v_1, v_2, \dots, v_n$, 则输出路径为

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \rightarrow v_1.$$

13. 共享单车随机抽样分配算法

设当前可用单车集合为 $B = \{b_1, b_2, \dots, b_N\}$, 用户请求位置为 r , 随机抽样数量为 s , 其中 $1 \leq s \leq N$ 。

算法 9 共享单车随机抽样分配算法

- 1: 从 B 中不放回地均匀随机抽取 s 辆单车，得到样本集合 R
 - 2: **for** 每辆 $b \in R$ **do**
 - 3: 计算距离 $d(b, r)$
 - 4: **end for**
 - 5: 选择 $b^* = \arg \min_{b \in R} d(b, r)$
 - 6: 将 b^* 分配给用户，并从可用集合中删除
-

该算法的基本思想是：不搜索全部 N 辆单车，而只在随机抽取的 s 辆单车中寻找最近者，从而用随机性降低计算量。

若计算一次距离需要 $O(1)$ 时间，则每次请求的运行时间为

$$O(s),$$

而全局精确搜索需要

$$O(N).$$

设按距离从近到远排序后，前 t 辆单车被认为是较好选择。随机样本中至少抽到一辆前 t 好单车的概率为

$$1 - \frac{\binom{N-t}{s}}{\binom{N}{s}}.$$

当 s 增大时，该概率增大，算法更可能选到距离较近的单车。当 $s = N$ 时，算法等价于搜索全部单车，一定得到最优选择。

因此，抽样数量越大，结果越接近最优，但运行时间越长；抽样数量越小，运行更快，但可能选到较远的单车。

14. 按到达顺序装箱算法及近似比

题目描述的是装箱问题中的 First-Fit 算法。设箱子容量为 1，包裹体积为 $a_1, a_2, \dots, a_n$ ，其中 $0 < a_i \leq 1$ 。

算法 10 First-Fit 装箱算法

```

1: 初始时箱子集合  $\mathcal{B} \leftarrow \emptyset$ 
2: for  $i = 1, 2, \dots, n$  do
3:    $placed \leftarrow false$ 
4:   for 按照打开时间从早到晚扫描每个箱子  $B \in \mathcal{B}$  do
5:     if  $B$  的剩余容量至少为  $a_i$  then
6:       将包裹  $a_i$  放入箱子  $B$ 
7:        $placed \leftarrow true$ 
8:       break
9:     end if
10:  end for
11:  if  $placed = false$  then
12:    打开一个新箱子  $B_{\text{new}}$ 
13:    将  $a_i$  放入  $B_{\text{new}}$ 
14:     $\mathcal{B} \leftarrow \mathcal{B} \cup \{B_{\text{new}}\}$ 
15:  end if
16: end for
17: return  $\mathcal{B}$ 

```

该算法可以看作近似算法，因为装箱问题要求使用最少箱子数，是困难的组合优化问题。First-Fit 不保证最优，但能在多项式时间内得到一个与最优解相差常数倍的可行解。

设 First-Fit 使用箱子数为 A ，最优算法使用箱子数为 OPT 。所有包裹总体积为

$$V = \sum_{i=1}^n a_i.$$

显然

$$\text{OPT} \geq V.$$

下面给出一个常数近似分析。First-Fit 的最终结果中，至多只有一个箱子的装载量不超过 $1/2$ 。否则，若存在两个装载量都不超过 $1/2$ 的箱子，设后打开的箱子为 B_2 ，其第一个包裹大小必然不超过 $1/2$ ；而较早箱子最终装载量不超过 $1/2$ ，在 B_2 打开时它的装载量也不超过 $1/2$ ，所以该包裹本可以放入较早箱子，与 First-Fit 的规则矛盾。

因此，除至多一个箱子外，其余箱子的装载量都大于 $1/2$ ，从而

$$V > \frac{A-1}{2}.$$

于是

$$A-1 < 2V \leq 2\text{OPT}.$$

所以

$$A < 2 \text{OPT} + 1.$$

也就是说，First-Fit 使用的箱子数至多约为最优箱子数的 2 倍，因此它是一个渐近意义下的

2-近似算法.